Aufgabe 1: Aktives Warten nach Peterson

Petersons Algorithmus für wechselseitigen Ausschluss – Formelsammlung & Übersicht

1. Zielsetzung & Problemstellung:
Garantie des wechselseitigen Ausschlusses (*Mutual Exclusion*) zweier konkurrierender Prozesse/Threads (P_0 und P_1) im User Space ohne spezielle atomare Hardware-Befehle (wie TSL, XCHG oder CAS).

2. Gemeinsame Variablen (*Shared State*):

- `bool interested[2] = {false, false};` – Signalisiert die Absicht eines Prozesses, den kritischen Abschnitt zu betreten.
- `int turn = 0;` – Höflichkeitsvariable zum Symmetriebruch: Bestimmt, wer im Konfliktfall warten muss.

3. Protokoll im Überblick:

Eintritt: Prozess P_0	**Eintritt: Prozess P_1**
`interested[0] = true;`	`interested[1] = true;`
`turn = 0; // überlässt P1 den Vortritt`	`turn = 1; // überlässt P0 den Vortritt`
`while (interested[1] && turn == 0); // warten`	`while (interested[0] && turn == 1); // warten`
`// Kritischer Abschnitt`	`// Kritischer Abschnitt`
`interested[0] = false; // Austritt`	`interested[1] = false; // Austritt`

Kernprinzip: Jeder Prozess setzt bei Eintritt `turn` auf sich selbst (Höflichkeitsprinzip). Wer als letzter schreibt, muss warten!

Simulator ↗

1.) Standard-Ablauf & Zeitlich versetzter Start

Prozess P_0 betritt zum Zeitpunkt t_0 als erster den kritischen Abschnitt. Während P_0 rechnet, möchte Prozess P_1 zum Zeitpunkt t_1 ebenfalls eintreten und ruft das Eintrittsprotokoll auf.

Aufgaben:

1. Welche Werte haben die Variablen `interested[0]`, `interested[1]` und `turn` zum Zeitpunkt t_1?
2. Warum tritt Prozess P_1 in die `while`-Warteschleife ein?
3. Was passiert genau, wenn Prozess P_0 den kritischen Abschnitt verlässt?


Simulator ↗

2.) Symmetriebruch bei zeitgleichem Eintreffen

Nehmen Sie an, Prozess P_0 und Prozess P_1 rufen exakt zeitgleich das Eintrittsprotokoll auf und beide setzen ihr jeweiliges `interested`-Flag auf `true`.

Aufgaben:

1. Warum entsteht hierbei **keine** Verklemmung (*Deadlock*)?
2. Welcher Prozess darf als erster in den kritischen Abschnitt eintreten, wenn Prozess P_1 den Schreibzugriff auf `turn` einen Nanosekunden-Bruchteil nach Prozess P_0 ausführt?
3. Begründen Sie die Entscheidung anhand der booleschen Schleifenbedingungen.

Simulator ↗

3.) Fehleranalyse: Weglassen der Höflichkeitsvariable `turn`

Ein Entwickler möchte den Code vereinfachen und streicht die Zuweisung von `turn`:

Fehlerhafter Eintritt P_0	**Fehlerhafter Eintritt P_1**
`interested[0] = true;`	`interested[1] = true;`
`while (interested[1]); // warten`	`while (interested[0]); // warten`

Aufgaben:

1. Zu welchem schwerwiegenden Synchronisationsfehler führt dieser fehlerhafte Code?
2. Konstruieren Sie ein konkretes zeitliches Interleaving zweier Prozesse, bei dem dieser Fehler auftritt.
3. Welche der vier Tanenbaum-Bedingungen für kritische Abschnitte wird dadurch verletzt?

Simulator ↗

4.) Fehleranalyse: Vertauschte Reihenfolge von Zuweisungen

Ein Programmierer tauscht in `enter_region` versehentlich die Zuweisungen um:

Vertauschter Eintritt P_0	**Vertauschter Eintritt P_1**
`turn = 0; // Erst turn setzen!`	`turn = 1; // Erst turn setzen!`
`interested[0] = true; // Dann Flag!`	`interested[1] = true; // Dann Flag!`
`while (interested[1] && turn == 0);`	`while (interested[0] && turn == 1);`

Aufgaben:

1. Zeigen Sie durch ein konkretes Interleaving, dass dadurch der **wechselseitige Ausschluss verletzt** wird!
2. Beschreiben Sie den Zustand beider Prozesse am Ende Ihres Interleavings.

Simulator ↗

5.) Realität moderner Multicore-Hardware & Memory Barriers

In der Theorie gilt der Peterson-Algorithmus als mathematisch bewiesen korrekt. Auf modernen Mehrkernprozessoren (z. B. x86, ARM oder Apple Silicon) führt eine naive C-Implementierung jedoch im praktischen Stresstest reproduzierbar zu Race Conditions.

Aufgaben:

1. Welcher Hardware-Mechanismus moderner CPUs (bzw. Optimierung moderner Compiler) verursacht dieses Phänomen?
2. Wie muss der Code auf modernen Plattformen geschützt werden, um die Korrektheit wiederherzustellen?

Petersons Algorithmus für wechselseitigen Ausschluss – Formelsammlung & Übersicht

1. Zielsetzung & Problemstellung:
Garantie des wechselseitigen Ausschlusses (*Mutual Exclusion*) zweier konkurrierender Prozesse/Threads (P_0 und P_1) im User Space ohne spezielle atomare Hardware-Befehle (wie TSL, XCHG oder CAS).

2. Gemeinsame Variablen (*Shared State*):

- `bool interested[2] = {false, false};` – Signalisiert die Absicht eines Prozesses, den kritischen Abschnitt zu betreten.
- `int turn = 0;` – Höflichkeitsvariable zum Symmetriebruch: Bestimmt, wer im Konfliktfall warten muss.

3. Protokoll im Überblick:

Eintritt: Prozess P_0	**Eintritt: Prozess P_1**
`interested[0] = true;`	`interested[1] = true;`
`turn = 0; // überlässt P1 den Vortritt`	`turn = 1; // überlässt P0 den Vortritt`
`while (interested[1] && turn == 0); // warten`	`while (interested[0] && turn == 1); // warten`
`// Kritischer Abschnitt`	`// Kritischer Abschnitt`
`interested[0] = false; // Austritt`	`interested[1] = false; // Austritt`

Kernprinzip: Jeder Prozess setzt bei Eintritt `turn` auf sich selbst (Höflichkeitsprinzip). Wer als letzter schreibt, muss warten!

Simulator ↗

1.) Standard-Ablauf & Zeitlich versetzter Start

Prozess P_0 betritt zum Zeitpunkt t_0 als erster den kritischen Abschnitt. Während P_0 rechnet, möchte Prozess P_1 zum Zeitpunkt t_1 ebenfalls eintreten und ruft das Eintrittsprotokoll auf.

Aufgaben:

1. Welche Werte haben die Variablen `interested[0]`, `interested[1]` und `turn` zum Zeitpunkt t_1?
2. Warum tritt Prozess P_1 in die `while`-Warteschleife ein?
3. Was passiert genau, wenn Prozess P_0 den kritischen Abschnitt verlässt?

Musterlösung Szenario 1:

1. **Variablenwerte bei t_1:**
   - `interested[0] = true` (P_0 rechnet im kritischen Abschnitt).
   - `interested[1] = true` (P_1 meldet Eintrittsabsicht an).
   - `turn = 1` (P_1 hat als letzter `turn = 1` zugewiesen).

2. **Warteschleife für P_1:** Bedingung: `interested[0] && turn == 1`.
Da P_0 noch aktiv ist (`interested[0] == true`) und `turn == 1` gilt, ist die Bedingung erfüllt. P_1 verbleibt im aktiven Warten (*Spinlock*).

3. **Austritt von P_0:** P_0 setzt `interested[0] = false`. Beim nächsten Prüfen der Schleifenbedingung durch P_1 ist `interested[0]` falsch. Die Schleife terminiert und P_1 betritt sofort den kritischen Abschnitt.

Simulator ↗

2.) Symmetriebruch bei zeitgleichem Eintreffen

Nehmen Sie an, Prozess P_0 und Prozess P_1 rufen exakt zeitgleich das Eintrittsprotokoll auf und beide setzen ihr jeweiliges `interested`-Flag auf `true`.

Aufgaben:

1. Warum entsteht hierbei **keine** Verklemmung (*Deadlock*)?
2. Welcher Prozess darf als erster in den kritischen Abschnitt eintreten, wenn Prozess P_1 den Schreibzugriff auf `turn` einen Nanosekunden-Bruchteil nach Prozess P_0 ausführt?
3. Begründen Sie die Entscheidung anhand der booleschen Schleifenbedingungen.

Musterlösung Szenario 2:

1. **Kein Deadlock:** Die Variable `turn` ist ein atomarer Skalar und kann zu jedem Zeitpunkt im Speicher nur genau **einen** Wert (0 oder 1) annehmen. Da P_0 nur bei `turn` $== 0$ und P_1 nur bei `turn` $== 1$ wartet, kann niemals für beide Prozesse gleichzeitig die Schleifenbedingung wahr

sein.

2. & 3. **Entscheidung bei Schreibfolge $\mathrm{P}_0 \rightarrow \mathrm{P}_1$:**

- P_0 schreibt zuerst: `turn = 0`.
- P_1 schreibt danach: `turn = 1` (überschreibt den Wert von P_0).
- **Prüfung P_0:** `interested[1] && turn == 0` $\Longrightarrow$ Da `turn` den Wert 1 hat, ist `turn` $== 0$ **falsch**! P_0 wartet nicht und betritt den kritischen Abschnitt.
- **Prüfung P_1:** `interested[0] && turn == 1` $\Longrightarrow$ Da `turn` $== 1$ und `interested[0]` $==$ `true`, ist die Bedingung **wahr**. P_1 wartet.

Ergebnis: Wer zuletzt `turn` zuweist, überlässt dem anderen den Vortritt. Hier gewinnt Prozess P_0!

Simulator ↗

3.) Fehleranalyse: Weglassen der Höflichkeitsvariable `turn`

Ein Entwickler möchte den Code vereinfachen und streicht die Zuweisung von `turn`:

Fehlerhafter Eintritt P_0	Fehlerhafter Eintritt P_1
`interested[0] = true;`	`interested[1] = true;`
`while (interested[1]); // warten`	`while (interested[0]); // warten`

Aufgaben:

1. Zu welchem schwerwiegenden Synchronisationsfehler führt dieser fehlerhafte Code?
2. Konstruieren Sie ein konkretes zeitliches Interleaving zweier Prozesse, bei dem dieser Fehler auftritt.
3. Welche der vier Tanenbaum-Bedingungen für kritische Abschnitte wird dadurch verletzt?

Musterlösung Szenario 3:

1. **Fehler:** Es entsteht eine vollständige **Verklemmung (Deadlock / Livelock)**.

2. **Konkretes Interleaving:**

   - t_0: Prozess P_0 führt `interested[0] = true;` aus.
   - t_1: Prozess P_1 führt `interested[1] = true;` aus.
   - t_2: Prozess P_0 prüft `while(interested[1])` $\Longrightarrow$ `true` (wartet).
   - t_3: Prozess P_1 prüft `while(interested[0])` $\Longrightarrow$ `true` (wartet).

   Beide Prozesse warten dauerhaft aufeinander. Keiner von beiden kann jemals den kritischen Abschnitt oder das Austrittsprotokoll erreichen.

3. **Verletzte Bedingung nach Tanenbaum:** Bedingung 4 (Kein Prozess darf unendlich lange warten müssen / Bounded Waiting) sowie Bedingung 3 (Fortschrittsgarantie: Außerhalb des kritischen Abschnitts darf kein Prozess blockiert werden).

Simulator ↗

4.) Fehleranalyse: Vertauschte Reihenfolge von Zuweisungen

Ein Programmierer tauscht in `enter_region` versehentlich die Zuweisungen um:

Vertauschter Eintritt P_0	Vertauschter Eintritt P_1
`turn = 0; // Erst turn setzen!`	`turn = 1; // Erst turn setzen!`
`interested[0] = true; // Dann Flag!`	`interested[1] = true; // Dann Flag!`
`while (interested[1] && turn == 0);`	`while (interested[0] && turn == 1);`

Aufgaben:

1. Zeigen Sie durch ein konkretes Interleaving, dass dadurch der **wechselseitige Ausschluss verletzt** wird!
2. Beschreiben Sie den Zustand beider Prozesse am Ende Ihres Interleavings.

Musterlösung Szenario 4:

1. **Interleaving zur Verletzung des wechselseitigen Ausschlusses:**

- t_0: Prozess P_0 führt `turn = 0;` aus.
- t_1: Prozess P_0 wird unterbrochen (Preemption), **bevor** er `interested[0] = true;` ausführen kann.
- t_2: Prozess P_1 läuft an: setzt `turn = 1;` und `interested[1] = true;`.
- t_3: Prozess P_1 prüft: `while (interested[0] && turn == 1)`. Da `interested[0]` noch `false` ist, schlägt die Bedingung fehl. $\Longrightarrow$ P_1 **betritt den kritischen Abschnitt!**
- t_4: Prozess P_0 wird reaktiviert und setzt nun `interested[0] = true;`.
- t_5: Prozess P_0 prüft: `while (interested[1] && turn == 0)`. Da `turn` in Schritt t_2 auf 1 gesetzt wurde, ist `turn` $== 0$ **falsch**! $\Longrightarrow$ P_0 **betritt EBENFALLS den kritischen Abschnitt!**

2. **Ergebnis:** Beide Prozesse befinden sich gleichzeitig im kritischen Abschnitt. Der wechselseitige Ausschluss (*Mutual Exclusion*) ist fundamental verletzt!

Simulator ↗

5.) Realität moderner Multicore-Hardware & Memory Barriers

In der Theorie gilt der Peterson-Algorithmus als mathematisch bewiesen korrekt. Auf modernen Mehrkernprozessoren (z. B. x86, ARM oder Apple Silicon) führt eine naive C-Implementierung jedoch im praktischen Stresstest reproduzierbar zu Race Conditions.

Aufgaben:

1. Welcher Hardware-Mechanismus moderner CPUs (bzw. Optimierung moderner Compiler) verursacht dieses Phänomen?
2. Wie muss der Code auf modernen Plattformen geschützt werden, um die Korrektheit wiederherzustellen?

Musterlösung Szenario 5:

1. **Ursache: Out-of-Order Execution & Speicherumordnung (Memory Reordering):**
   - Moderne CPUs verfügen über Caches und sogenannte *Store Buffers*. Um Latenzen zu minimieren, dürfen Speicherzugriffe auf unterschiedliche Adressen umgeordnet werden (*Relaxed Memory Consistency*, insb. Store-Load-Reordering).
   - Die CPU darf den Lesevorgang `interested[other]` in der Schleife vorziehen, bevor der Schreibvorgang `interested[process] = true` in den gemeinsamen Hauptspeicher propagiert wurde.
   - Aus Sicht beider Kerne erscheint der andere Prozess noch als unbeteiligt $\Longrightarrow$ beide Kerne treten simultan ein.
2. **Abhilfe & Schutzmaßnahmen:**
   - **Memory Barriers / Fences:** Einfügen expliziter Speicherbarrieren (z. B. `mfence` auf x86, `dmb` auf ARM, oder `Thread.MemoryBarrier()` in C#), die erzwingen, dass alle Schreibzugriffe vor nachfolgenden Lesezugriffen global sichtbar sind.
   - **Sequentielle Konsistenz:** Verwendung von C++11 `std::atomic` mit `std::memory_order_seq_cst` oder C# `volatile`.